Java语言基础

第二章.pdf

第二章

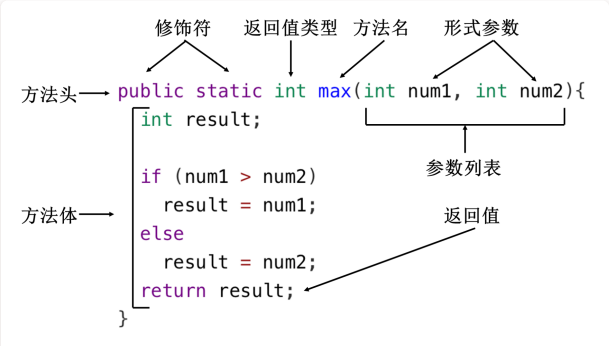
- 标识符、关键字、分隔符、注释
- Java基本数据类型、常量
- 控制流程
- 输入与输出
- 数组
- 异常机制
- Java虚拟机与垃圾回收

标识符、关键字、分隔符、注释

标识符的规定

1. 可由英文字母、数字、下划线、美元符号$组合而成，长度不受限制。
2. 必须以英文字母、下划线、美元符号$开头。
3. **区分字母大小写**

源代码的构成部分



一个方法由两部分组成：方法头和方法体

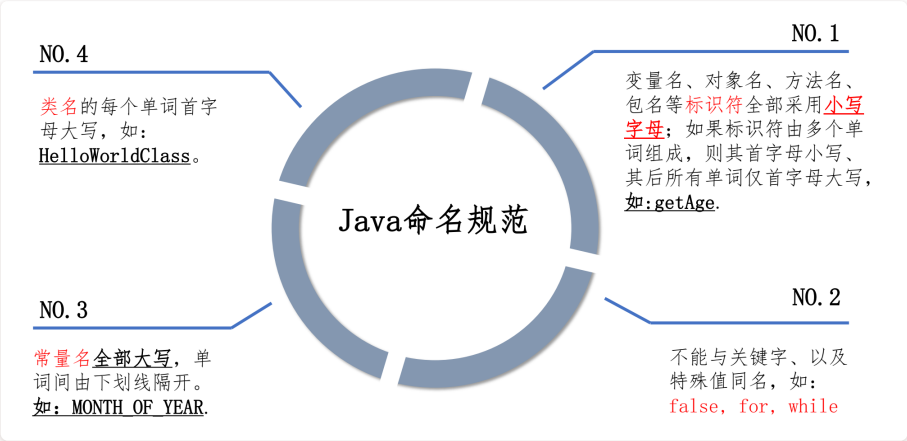
其中，方法头包括：

1. 修饰符
2. 返回值类型
3. 方法名
4. 形式参数（其集合叫参数列表）

标识符

标识符：用来标识类名、变量名、方法名、对象名、数组名、文件名的有效字符系列

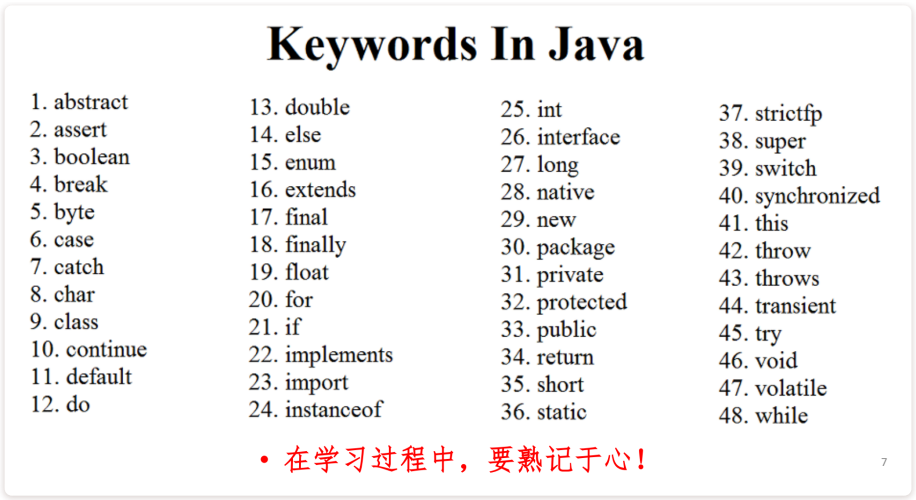
命名规范：



标识符命名规范：

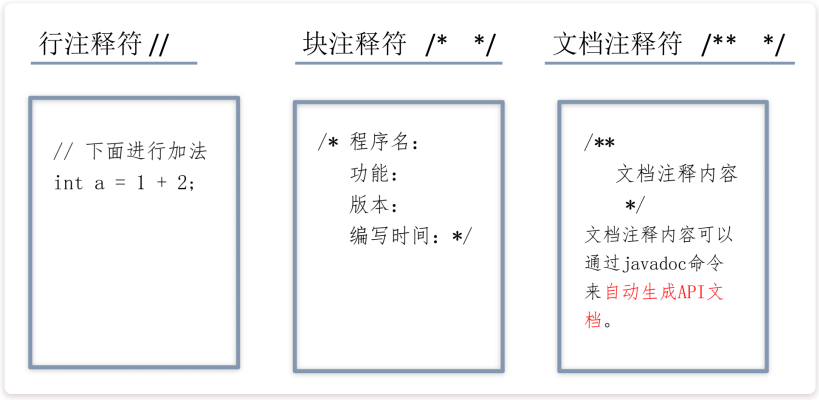
1. 变量对象方法包名均小写，驼峰法
2. 不能与关键字和特殊值同名
3. 常量名全部大写，单词之间用下划线隔开
4. 类名的首字母大写

关键字



分隔符





注意，如果要用数组，则需要在数组的基类后面加中括号

文档注释符的开头是/**，结尾和块注释符一样是*/，其中的内容可以通过javadoc命令来自动生成API文档

JAVA基本数据类型、常量

符号常量

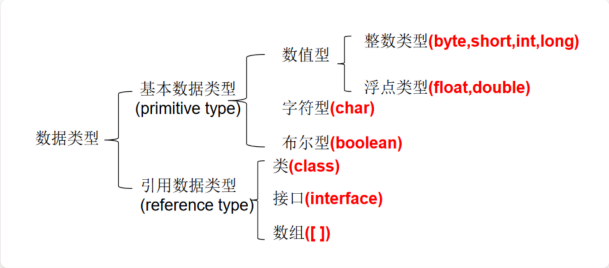
声明规则：

[修饰符] final 类型标识符 常量名 = 常量;

Java中有八种基本数据类型：

1. 数值型——整数类型byte short int long
2. 浮点类型——float double
3. 字符类型——char
4. 布尔类型——boolean

三种引用数据类型：class interface 数组



整数类型：byte, short, int, long

java整型常量默认为int型，声明long类型常量必须在末尾加上l或L

byte到long：1、2、4、8字节，分别代表8、16、32、64位

怎么求表数范围？首先注意有一位符号位，假设有n位

那么范围为$[-2^{n-1},2^{n-1}-1]$即为表数范围

类型	占用存储空间	表数范围
byte	1字节=8bit位	-128 ~ 127
short	2字节	-2^15~ 2^15-1
int	4字节	-2^31~ 2^31-1 (约21亿)
long	8字节	-2^63~ 2^63-1

字符类型：char

Java语言规范规定，Java里的char类型是UTF-16的code unit，即16位（2字节）

字符型变量的三种表现形式：

- 单引号括起来，char c1 = 'a'; char c2 = '9'
- 允许使用转义字符\来将其后的字符转变成特殊的字符型常量：char c1 = '\n'
- 直接使用Unicode值表示字符型常量：'\uXXXX'。其中XXXX代表一个十六进制整数

char类型是可以进行**运算**的，因为它都对应有Unicode码

布尔类型：boolean

一般用于流程控制

布尔类型只允许取值true和false，不可以用0或非0的值代替

浮点类型

Java的浮点类型采用IEEE浮点格式，有固定的表数范围和字段长度，不受具体操作系统的影响

Java的浮点类型常量默认为double型，声明float型常量，须在数字末尾加f或F

类型	占用存储空间	表数范围
单精度float	4字节	-3.403E38 ~ 3.403E38 （有效位数6-7位）
双精度double	8字节	-1.798E308 ~ 1.798E308（有效位数为15位）

这里记住float和double分别是4、8字节

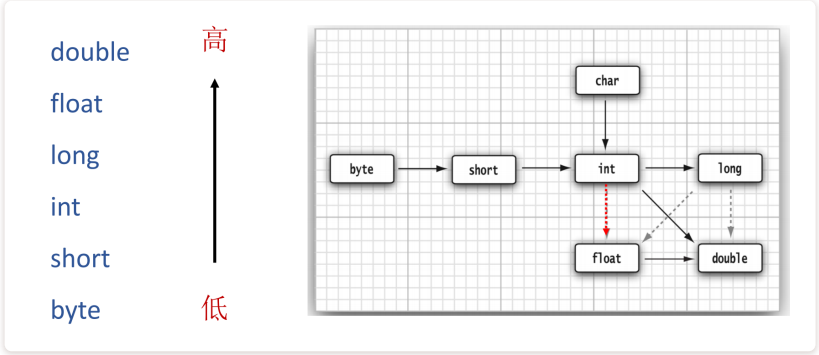
使用末尾加f或者用new Float()的方法来定义浮点数

```java
//Java浮点类型构造方法
//正确示例
float f = 3.14f;
float f = new Float(3.14);

//错误示例
float f = 3.14;
```

不同类型数据间的转换

数据类型的精度排名：



类型转换规则

1. 低级别转高级别，自动完成

```java
float x=200; //将int类型值200转换成float类型值200.0
```

2. 高级别转低级别，必须进行**强制类型转换**

```java
int i;
i = (int) 26L; //将long类型值26转换成int类型值26.
```

3. 强制类型转换时，可能造成数据精度的丢失

字符串数字转换为数值

注意这里讨论的是字符串转为数值,不是简单的强转!!!而且字符串必须得能转的过去，比如"123"确实能转成数字123，但是"123dashabi"是万万不可转化成数值的

1. 转换为整型示例：
   使用Integer类的parseInt()方法

```
int x1=Interger.parseInt("260");//字符串直接转换
int x2=Interger.parseInt(txt1.getText());//将txt1文本中的文本转换成int类型赋值给x2
```

2. 转换为浮点型示例：
   类似的，使用Float类的parseFloat方法或Double类的parseDouble类方法

```
float y=Float.parseFloat("23.5");
float z=Double.parseDouble("45.6");
```

常量与变量



注意要点：

1. 八进制开头要有0，十六进制开头有0X或0x

修饰符：

1. public
2. private
3. protected
4. 缺省

```
public final static Color black = new Color(0,0,0);
public final static Color BLACK= black;
public final static double PI = 3.1415926;
```

控制流程

条件语句

if条件

```
int a = 100;
if(a == 100)
{
        System.out.println(a);
}
```

if...else条件

```
int(math >= 60) { // 判断条件：math大于等于60成立
        System.out.println("math has passed");
} else { // 判断条件不满足
        System.out.println("math has not passed");
}
```

if...else if条件

```
if(x > 60) {
        System.out.println("x大于60");
} else if (x > 30) {
        System.out.println("x大于30但小于等于60");
} else if (x > 0) {
        System.out.println("x大于0但小于等于30");
} else {
        System.out.println("x小于等于0");
}
```

switch条件语句

```
switch (表达式) {
        case 常量值1:
                语句块1
                [break;]
        ...
        case 常量值n:
                语句块n
                [break;]
        default:
                语句块 n+1;
                [break;]
}
```

switch注意两点：

1. case+常量值之后要加冒号：
2. 每一个case结束之后要加break;，但是如果是switch块结尾的case或者是default则不需要加break;

循环语句

while

```
while（x <= 10) {
    sum += x;
    x++;
}
```

do...while

```
do {
    System.out.println("b == " + b);
    b--;
}while(b == 8)
```

至少执行一次

for

```
for(表达式1; 表达式2; 表达式3)
{
    语句序列
}
```

跳转语句

break

退出当前循环，注意如果两层循环，则**只能跳出内层循环**

continue

跳过后面的语句，进行下一次循环

return

从一个方法返回，并将控制权交给调用它的语句

输入与输出

标准输入System.in

Q: 什么是System.in?
A: 一个InputStream类的对象

使用方法：

1. 使用字符流对System.in封装

```
BufferedReader stdin = new BufferedReader(new InputStreamReader(System.in));
System.out.print("Enter a line: ");
System.out.println(stdin.readLine());
```

这里首先定义一个BufferedReader对象，和声明一个对象类似，=new 构造方法（），在构造方法里再写一个字节流转字符流的工具InputStreamReader()，这里再把System.in封装起来，然后可以使用stdin.readLine()方法来读取用户输入的一行，注意如果用户没有输入则会等待用户输入，注意这个步骤需要导入java.io包：

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
```

2. 使用java.util.Scanner对System.in进行封装

这种方法更简单，只需要定义一个Scanner对象对System.in封装即可，然后只需要调用.nextLine()方法

```
Scanner stdin = new Scanner(System.in);
System.out.print("Enter a line: );
System.out.println(stdin.nextLine());
```

标准输出System.out

System.out是一个PrintStream类的对象，可以调用print、println或write成员方法来在控制台输出各种数据

print和println区别只在于：**后者输出后换行而print不换行**

write方法用来输出**字节数组**，输出时**不换行**

数组

声明数组

```java
int[] a;
//Java提倡的

int b[];
//C风格
```

创建数组

```java
int[] c = new int[2];
//指定数组长度

int[] c = new int[]{0,1};
//直接给数组赋值
```

> 注意，同时使用维表达式和初始化创建数组是非法的，如下面所示：
>
> int [] c = new int[2]{0,1};

查看数组类型

```java
int[] a = new int[3];
System.out.println(a.getClass().getName());

//输出：[I
```

这里使用了反射，用getClass方法得到a的类型，然后用getName获取类型名，值得一提的是，getClass只能对Object类使用，如果对基本数据类型int使用会报错：

```java
public static void main(String[] args) {
    int a = Integer.parseInt( s: 2);
    System.out.println(a.getClass().getName());
}
```

数组访问

数组长度

a.length返回一个数组的长度

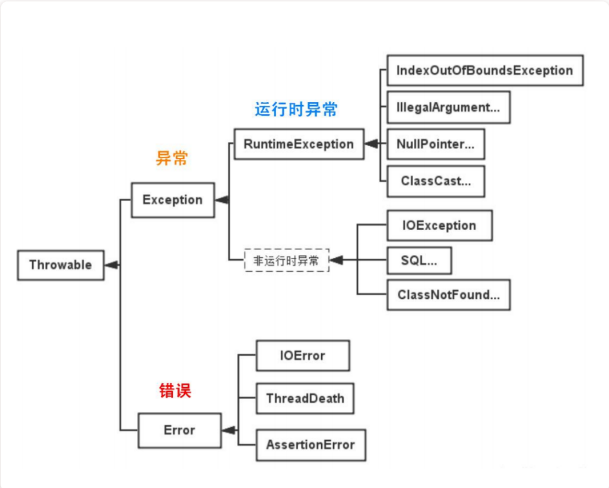
注意这里的length不需要加()，可以理解为Interger[]类的一个属性

数组越界

数组访问下标的范围是0到长度-1

异常机制

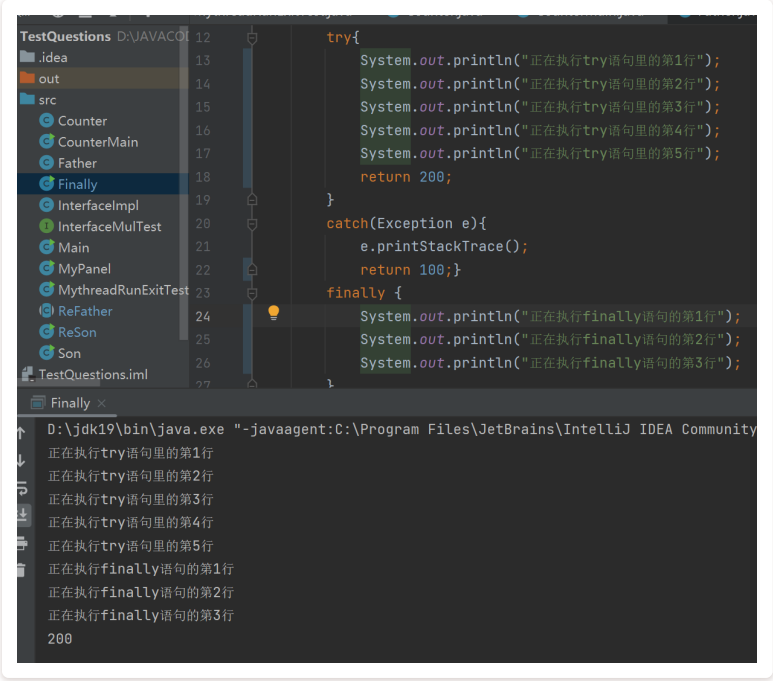
异常机制是Java提供的一种识别和响应错误的一致性机制，Java异常机制可以使程序中异常代码和正常业务代码分离，保证程序的代码更加优雅



> 异常是指在程序执行过程中出现的一种**可预见的**、**可处理**的问题。异常分为编译时异常（Checked Exception）和运行时异常（Unchecked Exception）。编译时异常是指在编译期间就可以检测到的异常，需要在代码中进行捕获处理，否则程序无法通过编译。运行时异常是指在程序执行期间可能会发生的异常，通常由程序逻辑错误或者运行时环境导致，可以通过捕获处理来避免程序崩溃。
>
> 错误是指程序在运行时出现的一种**无法恢复**的问题。错误通常是由系统级别的问题或者内存不足等情况导致的。与异常不同，错误一般不应该被捕获处理，因为程序无法恢复正常运行。当出现错误时，程序会直接终止运行。

异常关键字

- try-用于监听，将被监听的代码放在try语句块里，出现异常则抛出异常
- catch-捕获异常，**注意其后要加括号**！
- finally-总是会被执行，主要用于回收在try里打开的物力资源，只有finally执行完了之后才会执行try或者catch块中的return或者throw语句，如果finally中使用了return或者throw等终止方法的语句，则不会跳回执行，直接停止



- throws-用在**方法签名**中，用于**声明该方法可能抛出的异常**

> 在Java中，方法签名（Method Signature）指的是一个方法的名称、参数类型以及参数顺序的组合。方法签名不包括方法的返回类型和访问修饰符。

> 下面是一个方法签名的示例：
>
> ```
> public void myMethod(String arg1, int arg2)
> ```
>
> 在这个例子中，方法签名是`myMethod(String, int)`。注意，方法签名不包括方法体的具体实现，只是定义了方法的参数类型、顺序和名称

- throw-用于抛出一个语句异常

异常的捕获处理方式

try-catch

在一个 try-catch 语句块中可以捕获多个异常类型，并对不同类型的异常做出不同的处理

```
try{
        //程序代码
} catch(Exception1 e1){
        //捕获异常Exception1并处理
} catch(Exception2 e2){
        //捕获异常Exception2并处理
}
...
```

try-finally

try块中引起异常，异常代码之后的语句不再执行，直接执行finally语句。try块没有引发异常，则执行完try块就执行finally语句。

```
try{
// 执行程序代码，可能会出现异常
} finally {
// 一定会执行的代码
...
```

try-catch-finally

```
try{
// 执行程序代码，可能会出现异常
} catch(Exception e) {
// 捕获到Exception异常并处理
} finally {
// 一定会执行的代码
}
...
```

throw和throws的用法

共同点

只负责抛出异常，不处理

用户程序自定义的异常和应用程序特定的异常，必须借助throws和throw来定义抛出异常

区别

- throws用于**方法头**，表示的只是**异常的申明**，而throw用于**方法内部**，语句**抛出一个异常**。
- throws可以一次性抛出多个异常，而throw只能一个。

用法

```
public static int getElement(int[] arr, int index) throws Exception,IOExcepti
on {
    if(arr==null) {
        throw new NullPointerException("指针为空");
    }else if(index>=arr.length-1) {
        throw new ArrayIndexOutOfBoundsException("数组越界");
    }
    int ele = arr[index];

```

```
    return ele;
}
```

怎么throw

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.util.Scanner;

public class Test {
    public static void main(String[] args) {
        try{
            System.out.println("1");
            System.out.println("2");
            if(true) {
                throw new Error("故意的");
            }
            System.out.println("3");
            System.out.println("4");
            System.out.println("5");
        }
        catch(Exception e){
            e.printStackTrace();
        }
        finally {
            System.out.println("这里是肯定会被执行的内容");
        }
    }
}
```

来自智者GPT：

> 在 Java 中，`throws` 关键字用于指定方法可能抛出的异常类型。当一个方法使用 `throws` 关键字声明一个异常时，它表明该方法可能会产生这种异常，但是并不负责处理该异常，而是将该异常传递给该方法的调用者来处理。
>
> `throws` 关键字通常在方法声明的尾部使用，例如：
>
> java Copy code
>
> ```java
> public void readFile(String fileName) throws IOException {
>     // code to read a file
> }
> ```
>
> 上面的例子中，`readFile` 方法可能会抛出 `IOException` 异常，当该方法被调用时，调用者需要考虑如何处理这种异常，可以使用 `try-catch` 块来捕获并处理异常。
>
> 使用 `throws` 关键字可以使程序员更好地了解方法的异常情况，可以让代码更加清晰和易于维护。同时，它也提供了一种处理异常的机制，使得程序可以更加健壮和可靠。

比如说，我们在使用序列化的时候，IDE通常会强制要我们写一个trycatch块，这就是序列化API里相关方法签名里使用throws，因此我们在尝试调用这个方法的时候必须考虑到如何处理这种异常

Java虚拟机与垃圾回收

JAVA虚拟机——JVM

JVM(Java Virtual Machine)

虚拟机是指软件通过模拟具有完整硬件系统的，运行在一个完全隔离环境中的计算机系统

JVM是通过软件来模拟JAVA字节码的指令集，是JAVA程序的运行环境

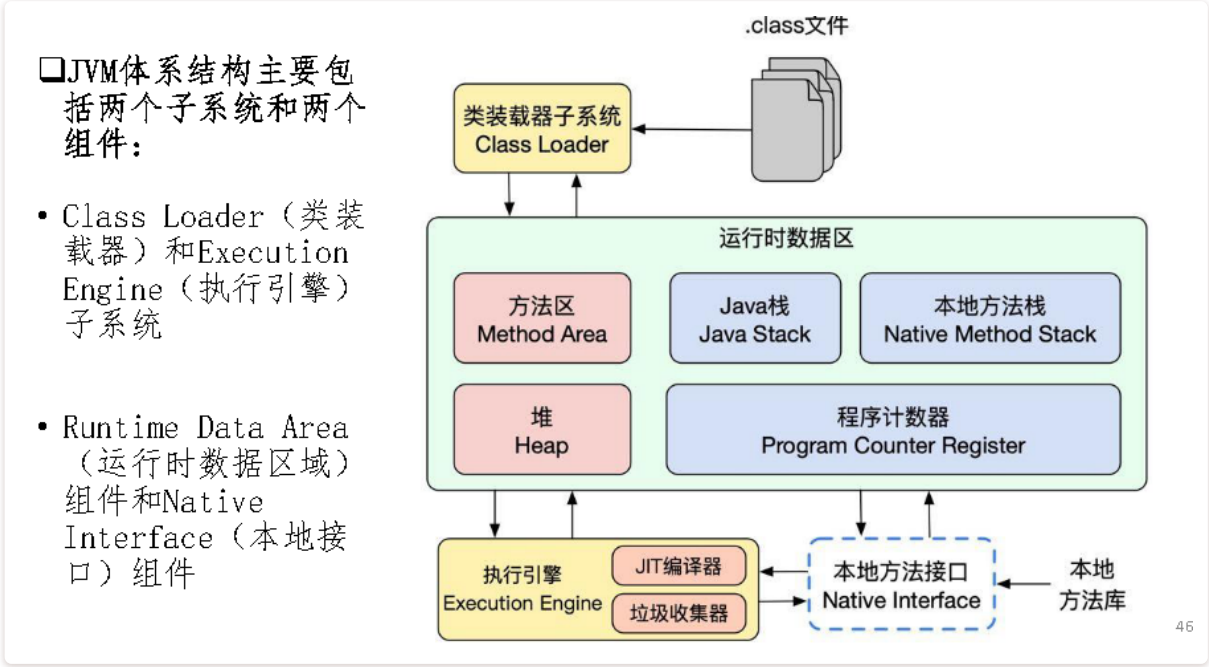
如今JVM也支持其他语言，比如Kotlin，转换的字节码文件都能通过java虚拟机进行运行和处理

特点

1. 一次编译，到处运行

2. 自动内存管理
3. 自动垃圾回收功能

JVM体系结构



JVM体系结构包括两个子系统和两个组件：

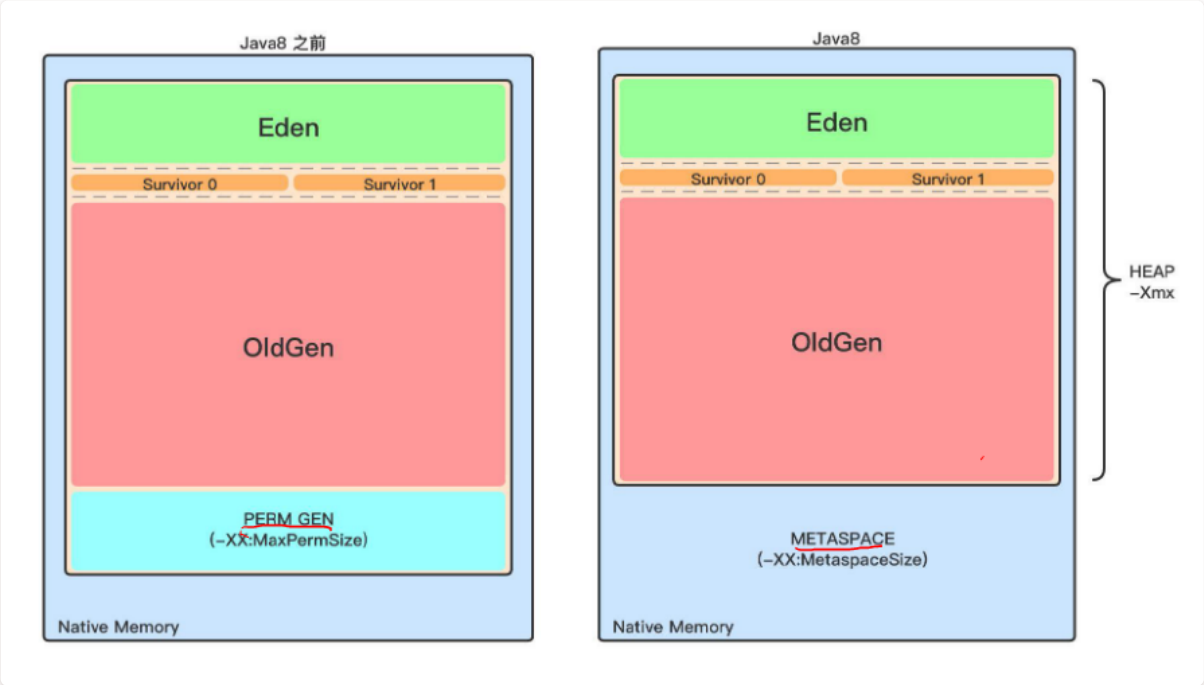
子系统：

1. Class Loader类装载器
2. Execution Engine执行引擎

组件：

1. Runtime Data Area运行时数据区域
2. Interface本地接口组件

JAVA堆



为了进行高效的垃圾回收，虚拟机把堆内存从逻辑上分成三块区域：

- 新生代（年轻代）：新对象和没达到一定年龄的对象都在新生代
- 老年代（养老区）：被长时间使用的对象，老年代的内存空间应该比年轻代大
- 元空间（JDK1.8之前叫永久代）：一些方法的操作临时对象等

JVM垃圾回收

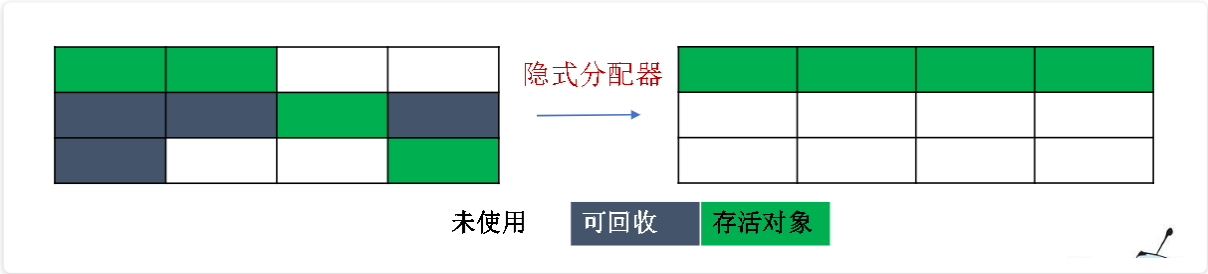
C和C++

C和C++采用显示分配器将堆空间暴露给用户，优点是牛逼的用户用的好，缺点是每次都要释放，不然就会内存泄漏

JVM垃圾回收

JAVA使用隐式分配器，程序员只管创建对象使用堆内存，回收交给垃圾回收器

JAVA垃圾回收期在执行引擎中，回收的主要对象是JVM堆空间



垃圾回收器的任务

- 跟踪监控每一个JAVA对象，一旦其处于不可达状态，回收该对象占用的内存
- 清理内存分配，清理回收过程中产生的内存碎片

优点

- 对开发者屏蔽了内存管理细节，提高了开发效率
- 开发者无权操作内存，减少了内存泄漏的风险

缺点

- 垃圾回收不受开发者控制，在对时间敏感的场景，不受控的垃圾回收会带来多余的时间开销

JVM、JRE、JDK的区别

JVM

Java虚拟机

所有JAVA程序运行在JVM上，是JRE的一部分

JRE

java runtime environment（java运行环境）

JRE主要用于执行JAVA程序，JRE除了包括JVM以外还包括一些基础的JAVA API，JRE是JDK的一部分

JDK

java development it（java开发工具）

提供了JAVA的开发环境和运行环境（JRE），开发环境主要包括了一些常用工具，比如编译工具打包程序等

